Linux Kernel Programming

Linux Kernel programming

Agenda

- Module Programming
- Procfs, Sysfs, Debugfs
- Sleep,delay
- Copy from user , copy to user
- Delayed exection
- Character device driver – Old Method
- LED Driver IOCTL

Module information

- MODULE_DESCRIPTION()
- MODULE_AUTHOR()
- MODULE_LICENSE()
- MODULE_VERSION();
- MODULE_INFO(my_field_name, "What easy value");

Kernel Module parameters

As a user program does, a kernel module can accept arguments from the command line. This allows dynamically changing the behavior of the module according to the given parameters, and can avoid the developer having to indefinitely change/compile the module during a test/debug session.

In order to set this up, you should first declare the variables that will hold the values of command line arguments, and use the module_param() macro on each of these. The macro is defined in include/linux/moduleparam.h (this should be included in the code too: #include <linux/moduleparam.h>), shown as follows:

module_param(name, type, perm);

This macro contains the following elements:

- **name**: The name of the variable used as the parameter
- **type**: The parameter's type (bool, charp, byte, short, ushort, int, uint, long, ulong), where charp stands for char pointer
- **perm**: This represents the /sys/module/<module>/parameters/<param> file permissions. Some of them are S_IWUSR, S_IRUSR, S_IXUSR, S_IRGRP, S_WGRP,and S_IRUGO

Kernel Module parameters

where:

S_I is just a prefix

R: read, W: write, X: execute

USR: user, GRP: group, UGO: user, group, others

One can eventually use a | (OR operation) to set multiple permissions. If perm is 0, the file parameter in sysfs will not be created. You should use only S_IRUGO read-only parameters, which I highly recommend; by making a | (OR) with other properties, you can obtain fine-grained properties.

When using module parameters, you should use MODULE_PARM_DESC in order to describe each of them. This macro will populate the module info section with each parameter's description.

Kernel Module parameters

```
#include <linux/moduleparam.h>
[...]

static char *mystr = "hello";
static int myint = 1;
static int myarr[3] = {0, 1, 2};

module_param(myint, int, S_IRUGO);
module_param(mystr, charp, S_IRUGO);
module_param_array(myarr, int,NULL, S_IWUSR|S_IRUSR); /*  */

MODULE_PARM_DESC(myint,"this is my int variable");
MODULE_PARM_DESC(mystr,"this is my char pointer variable");
MODULE_PARM_DESC(myarr,"this is my array of int");
```

Kernel Module parameters

```
static int foo()
{
    pr_info("mystring is a string: %s\n", mystr);
    pr_info("Array elements: %d\t%d\t%d", myarr[0], myarr[1], myarr[2]);
    return myint;
}
```

To load the module and feed our parameter, we do the following:

```
# insmod hellomodule-params.ko mystring=“test" myint=15 myArray=1,2,3
```

Kernel Module Handson1

sudo insmod example2.ko myint=11 mylong=22 myshort=45 myintArray=55,66

Kernel Module Handson2

Procfs,Debugfs, Sysfs

- There are three commonly used pseudo file systems in the kernel: procfs, debugfs, and sysfs.
- Procfs: The proc filesystem is a pseudo-filesystem which provides an interface to kernel data structures.
- Sysfs: The filesystem for exporting kernel objects.
- debugfs : Debugfs exists as a simple way for kernel developers to make information available to user space.

Procfs,Debugfs, Sysfs

They are used for data exchange between the Linux kernel and user space, but the applicable scenarios are different:

- The earliest history of procfs was originally used to interact with the kernel to obtain various information such as processors, memory, device drivers, and processes.

- Sysfs is closely tied to the kobject framework, and kobject exists for the device driver model, so sysfs is for device drivers.

- Debugfs is born from the name of the name, so it is more flexible.

Procfs,Debugfs, Sysfs

Their mounting methods are similar, let's do an experiment:

```
$ sudo mkdir /tmp/{proc,sys,debug}
$ sudo mount -t proc nondev /tmp/proc/
$ sudo mount -t sysfs nondev /tmp/sys/
$ sudo mount -t debugfs nondev /tmp/debug/
```

Procfs,Debugfs, Sysfs

The following is a brief introduction to the usage of these three file systems. Before you introduce, please write down their official documentation:

procfs — Documentation/filesystems/proc.txt

sysfs — Documentation/filesystems/sysfs.txt

debugfs — Documentation/filesystems/debugfs.txt

Debugfs

struct dentry *debugfs_create_dir(const char *name, struct dentry *parent)

struct dentry *debugfs_create_file(const char *name, umode_t mode, struct dentry *parent, void *data,const struct file_operations *fops)

Procfs

Many or most Linux users have at least heard of proc. Some of you may wonder why this folder is so important.

On the root, there is a folder titled “proc”. This folder is not really on /dev/sda1 or where ever you think the folder resides. This folder is a mount point for the procfs (Process Filesystem) which is a filesystem in memory. Many processes store information about themselves on this virtual filesystem. ProcFS also stores other system information.

It can act as a bridge connecting the user space and the kernel space. Userspace programs can use proc files to read the information exported by the kernel. Every entry in the proc file system provides some information from the kernel.

Procfs

The entry “meminfo” gives the details of the memory being used in the system.

To read the data in this entry just run

```
cat /proc/meminfo
```

Similarly the “modules” entry gives details of all the modules that are currently a part of the kernel.

```
cat /proc/modules
```

Procfs

It gives similar information as lsmod. Like this more, proc entries are there.

/proc/devices — registered character and block major numbers

/proc/iomem — on-system physical RAM and bus device addresses

/proc/ioports — on-system I/O port addresses (especially for x86 systems)

/proc/interrupts — registered interrupt request numbers

/proc/softirqs — registered soft IRQs

/proc/swaps — currently active swaps

/proc/kallsyms — running kernel symbols, including from loaded modules

/proc/partitions — currently connected block devices and their partitions

/proc/filesystems — currently active filesystem drivers

/proc/cpuinfo — information about the CPU(s) on the system

Procfs

Most proc files are read-only and only expose kernel information to user space programs.

proc files can also be used to control and modify kernel behavior on the fly. The proc files need to be writable in this case.

For example, to enable IP forwarding of iptable, one can use the command below,

echo 1 > /proc/sys/net/ipv4/ip_forward

The proc file system is also very useful when we want to debug a kernel module. While debugging we might want to know the values of various variables in the module or maybe the data that the module is handling. In such situations, we can create a proc entry for ourselves and dump whatever data we want to look into in the entry.

Procfs

The proc entry can also be used to pass data to the kernel by writing into the kernel, so there can be two kinds of proc entries.

1. An entry that only reads data from the kernel space.
2. An entry that reads as well as writes data into and from kernel space.

Procfs

You can create the directory under /proc/* using the below API.

```
struct proc_dir_entry *proc_mkdir(const char *name, struct proc_dir_entry
                                  *parent)
```

where,

`name`: The name of the directory that will be created under `/proc`.

`parent`: In case the folder needs to be created in a subfolder under `/proc` a pointer to the same is passed else it can be left as `NULL`.

Creating Procfs Entry

The creation of proc entries has undergone a considerable change in kernel version 3.10 and above. In this post, we will see one of the methods we can use in Linux kernel version 3.10. Let us see how we can create proc entries in version 3.10.

```
struct proc_dir_entry *proc_create ( const char *name, umode_t mode,
struct proc_dir_entry *parent, const struct file_operations *proc_fops )
```

sysfs

- Sysfs is a virtual filesystem exported by the kernel, similar to /proc. The files in Sysfs contain information about devices and drivers. Some files in Sysfs are even writable, for configuration and control of devices attached to the system. Sysfs is always mounted on /sys.

- The directories in Sysfs contain the hierarchy of devices, as they are attached to the computer.

- Sysfs is the commonly used method to export system information from the kernel space to the user space for specific devices. The sysfs is tied to the device driver model of the kernel. The procfs is used to export the process-specific information and the debugfs is used to use for exporting the debug information by the developer.

sysfs

Kernel Objects

The heart of the sysfs model is the kobject. Kobject is the glue that binds the sysfs and the kernel, which is represented by struct kobject and defined in <linux/kobject.h>. A struct kobject represents a kernel object, maybe a device or so, such as the things that show up as directory in the sysfs filesystem.

Kobjects are usually embedded in other structures.

It is defined as,

```
#define KOBJ_NAME_LEN 20

struct kobject {
 char *k_name;
 char name[KOBJ_NAME_LEN];
 struct kref kref;
 struct list_head entry;
 struct kobject *parent;
 struct kset *kset;
 struct kobj_type *ktype;
 struct dentry *dentry;
};
```

sysfs

Some of the important fields are:

struct kobject

|– name (Name of the kobject. Current kobject is created with this name in sysfs.)

|– parent (This is kobject's parent. When we create a directory in sysfs for the current kobject, it will create under this parent directory)

|– ktype (the type associated with a kobject)

|– kset (a group of kobjects all of which are embedded in structures of the same type)

|– sd (points to a sysfs_dirent structure that represents this kobject in sysfs.)

|– kref (provides reference counting)

It is the glue that holds much of the device model and its sysfs interface together.

So kobject is used to create kobject directory in /sys. This is enough. We will not go deep into the kobjects.

sysfs

There are several steps to creating and using sysfs.

1. Create a directory in `/sys`
2. Create Sysfs file

sysfs

Create a directory in /sys

We can use this function (`kobject_create_and_add`) to create a directory.

```
struct kobject * kobject_create_and_add ( const char * name, struct kobject
                                          * parent);
```

Where,

<name> – the name for the kobject

<parent> – the parent kobject of this kobject, if any.

If you pass `kernel_kobj` to the second argument, it will create the directory under /sys/kernel/. If you pass `firmware_kobj` to the second argument, it will create the directory under /sys/firmware/. If you pass `fs_kobj` to the second argument, it will create the directory under **/sys/fs/**. If you pass `NULL` to the second argument, it will create the directory under /sys/.

sysfs

To create a single file attribute we are going to use '`sysfs_create_file`'.

```
int sysfs_create_file ( struct kobject *  kobj, const struct attribute
* attr);
```

Where,

`kobj` - *object we're creating for.*

`attr` - attribute descriptor.

Once you have done with the sysfs file, you should delete this file using

`sysfs_remove_file`

```
void sysfs_remove_file  ( struct  kobject  *  kobj, const  struct  attribute
* attr);
```

Where,

`kobj` - *object we're creating for.*

`attr` - attribute descriptor.

Sleep

- Non-busy-wait alternatives for millisecond or longer delays

```
#include <linux/delay.h>

void msleep(unsigned int millisecs);
unsigned long msleep_interruptible(unsigned int millisecs);
void ssleep(unsigned int seconds);
```

- **msleep** and **ssleep** are not interruptible
- **msleep_interruptible** returns the remaining milliseconds

Delay

```
#include <linux/delay.h>

void ndelay(unsigned long nsecs); /* nanoseconds */
void udelay(unsigned long usecs); /* microseconds */
void mdelay(unsigned long msecs); /* milliseconds */
```

- Perform busy waiting

Copy Data to/from User space

unsigned long copy_from_user(void *to, const void __user *from,
unsigned long n)
unsigned long copy_to_user(void __user *to, const void *from,
unsigned long n)

In both cases, pointers prefixed with __user point to user space (untrusted) memory. n represents the number of bytes to copy. from represents the source address, and to is the destination address. Each of these returns the number of bytes that could not be copied. On success, the return value should be 0. Please do note that with copy_to_user(), if some data could not be copied, the function will pad the copied data to the requested size using zero bytes.

Copy Data to/from User space

When it comes to copying single and simple variables, such as char and int, but not larger data types, such as structures or arrays, the kernel offers dedicated macros in order to quickly perform the desired operation. These macros are put_user(x, ptr) and get_user(x, ptr), which are explained as follows:

put_user(x, ptr): This macro copies a variable from kernel space to user space. x represents the value to copy to user space, and ptr is the destination address in user space. The macro returns 0 on success, or -EFAULT on error. x

must be assignable to the result of dereferencing ptr. In other words, they must have (or point to) the same type.

get_user(x, ptr): This macro copies a variable from user space to kernel space, and returns 0 on success or -EFAULT on error. Please do note that x is set to 0 on error. x represents the kernel variable to store the result, and ptr is the source address in user space. The result of dereferencing ptr must be assignable to x without a cast. Guess what it means.

Delayed Exeution

Linux kernel keeps track of number of ticks since the kernel has booted in a global variable called jiffies. The number of ticks per second depends upon HZ which is defined to 100 in most of the systems, therefore jiffies is incremented every 10 ms.

Character Driver

- Character devices transfer data to or from a user application by means of characters, in a stream manner (one character after another), like a serial port does. A character device driver exposes the properties and functionalities of a device by means of a special file in the /dev directory, which you can use to exchange data between the device and user application, and which also allows you to control the real physical device. This is the basic concept of Linux that says everything is a file.

- A character device driver represents the most basic device driver in the kernel source.

Character Driver

- Kernel uses number-pairs (major,minor)
- The ‘major’ number identifies the driver
- The ‘minor’ number identifies the device
- One driver can control multiple devices
- Range for ‘major’ numbers is 0..255
- Certain of these values are ‘reserved’

Character Driver

- Driver-author can select a major number
- Kernel is told during driver ‘registration’
- But author must be careful: no duplication!
- Registration fails if number already used
- View currently used major numbers with

  $ cat /proc/devices
- Module author can let kernel choose major
- This is why major-number 0 is never used
- If programmer requests major-number 0,
- kernel assigns an available major-number
- Kernel informs driver during ‘registration’

Character Driver

```
int register_chrdev( unsigned int major,
                     const char *driver_name,
                     struct file_operations *fops );
```

Returns: major-number (or error-code)

- Using 0 as first argument (‘major’) tells
- kernel to pick an unused major-number

Character Driver

```
int register_chrdev( unsigned int major,
                     const char *driver_name,
                     struct file_operations *fops );
```

Returns: major-number (or error-code)

- Using 0 as first argument (‘major’) tells
- kernel to pick an unused major-number

```
# mknod /dev/led c 15 0
```

Character Driver

void unregister_chrdev(unsigned int major,

const char *name)

where,

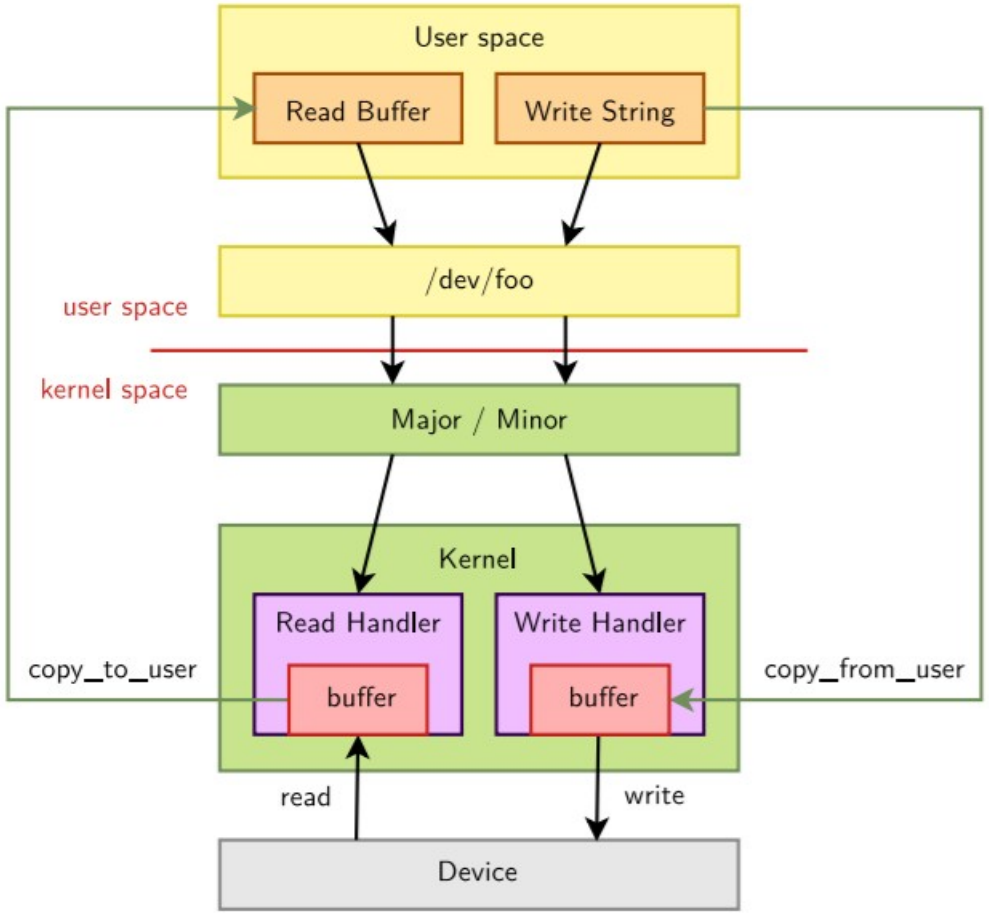
major: Major number of ddriver

name: name of the driver

Character Driver

- From the point of view of an application, a *character device* is essentially a **file**.
- The driver of a character device must therefore implement **operations** that let applications think the device is a file: `open`, `close`, `read`, `write`, etc.
- In order to achieve this, a character driver must implement the operations described in the `struct file_operations` structure and register them.
- The Linux filesystem layer will ensure that the driver's operations are called when a user space application makes the corresponding system call.

Character Driver



Character Driver

Here are the most important operations for a character driver, from the definition of `struct file_operations`:

```
struct file_operations {
    struct module *owner;
    ssize_t (*read) (struct file *, char __user *,
        size_t, loff_t *);
    ssize_t (*write) (struct file *, const char __user *,
        size_t, loff_t *);
    long (*unlocked_ioctl) (struct file *, unsigned int,
        unsigned long);
    int (*mmap) (struct file *, struct vm_area_struct *);
    int (*open) (struct inode *, struct file *);
    int (*release) (struct inode *, struct file *);
    ...
};
```

Many more operations exist. All of them are optional.

Character Driver

- `int foo_open(struct inode *i, struct file *f)`
  - Called when user space opens the device file.
  - **Only implement this function when you do something special with the device at `open()` time.**
  - `struct inode` is a structure that uniquely represents a file in the filesystem (be it a regular file, a directory, a symbolic link, a character or block device)
  - `struct file` is a structure created every time a file is opened. Several file structures can point to the same `inode` structure.
    - Contains information like the current position, the opening mode, etc.
    - Has a `void *private_data` pointer that one can freely use.
    - A pointer to the `file` structure is passed to all other operations
- `int foo_release(struct inode *i, struct file *f)`
  - Called when user space closes the file.
  - **Only implement this function when you do something special with the device at `close()` time.**

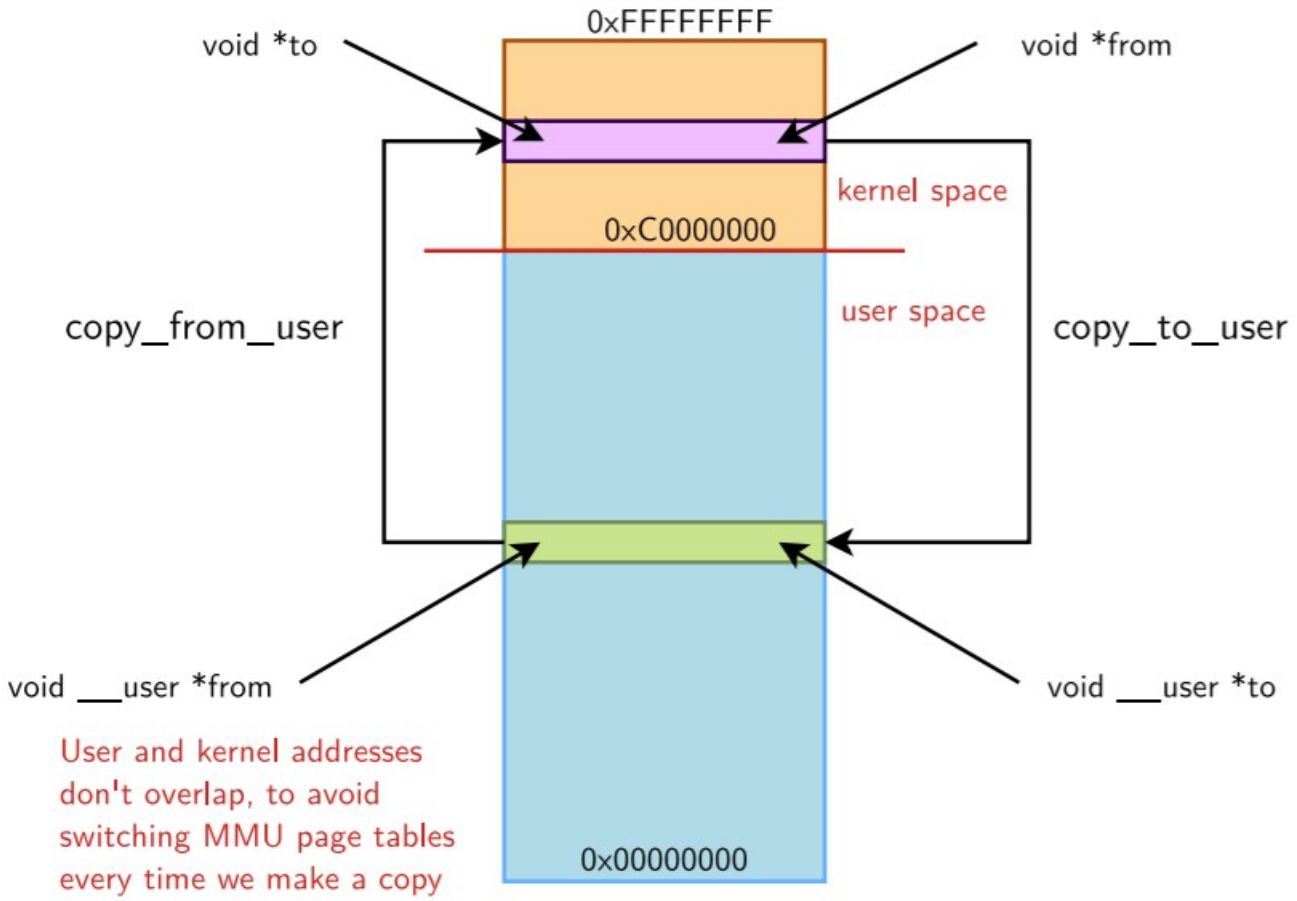
Character Driver

- `ssize_t foo_read(struct file *f, char __user *buf, size_t sz, loff_t *off)`
  - Called when user space uses the `read()` system call on the device.
  - Must read data from the device, write at most `sz` bytes to the user space buffer `buf`, and update the current position in the file `off`. `f` is a pointer to the same file structure that was passed in the `open()` operation
  - Must return the number of bytes read.
    `0` is usually interpreted by userspace as the end of the file.
  - On UNIX, `read()` operations typically block when there isn't enough data to read from the device
- `ssize_t foo_write(struct file *f, const char __user *buf, size_t sz, loff_t *off)`
  - Called when user space uses the `write()` system call on the device
  - The opposite of `read`, must read at most `sz` bytes from `buf`, write it to the device, update `off` and return the number of bytes written.

Character Driver

- Kernel code isn't allowed to directly access user space memory, using `memcpy()` or direct pointer dereferencing
  - Doing so does not work on some architectures
  - If the address passed by the application was invalid, the application would segfault.
  - **Never** trust user space. A malicious application could pass a kernel address which you could overwrite with device data (`read` case), or which you could dump to the device (`write` case).
- To keep the kernel code portable, secure, and have proper error handling, your driver must use special kernel functions to exchange data with user space.

Character Driver



Character Driver

- Having to copy data to or from an intermediate kernel buffer can become expensive when the amount of data to transfer is large (video).
- *Zero copy* options are possible:
  - `mmap()` system call to allow user space to directly access memory mapped I/O space. See our `mmap()` chapter.
  - `get_user_pages()` and related functions to get a mapping to user pages without having to copy them.

Character Driver

- `long unlocked_ioctl(struct file *f, unsigned int cmd, unsigned long arg)`
  - Associated to the `ioctl()` system call.
  - Called unlocked because it didn't hold the Big Kernel Lock (gone now).
  - Allows to extend the driver capabilities beyond the limited read/write API.
  - For example: changing the speed of a serial port, setting video output format, querying a device serial number... Used extensively in the V4L2 (video) and ALSA (sound) driver frameworks.
  - `cmd` is a number identifying the operation to perform.
    See `driver-api/ioctl` for the recommended way of choosing `cmd` numbers.
  - `arg` is the optional argument passed as third argument of the `ioctl()` system call.
    Can be an integer, an address, etc.
  - The semantic of `cmd` and `arg` is driver-specific.

Character Driver

```
static long phantom_ioctl(struct file *file, unsigned int cmd,
    unsigned long arg)
{
    struct phm_reg r;
    void __user *argp = (void __user *)arg;

    switch (cmd) {
    case PHN_SET_REG:
        if (copy_from_user(&r, argp, sizeof(r)))
            return -EFAULT;
        /* Do something */
        break;
    ...
    case PHN_GET_REG:
        if (copy_to_user(argp, &r, sizeof(r)))
            return -EFAULT;
        /* Do something */
        break;
    ...
    default:
        return -ENOTTY;
    }

    return 0;
}
```

Character Driver

```c
int main(void)
{
    int fd, ret;
    struct phm_reg reg;

    fd = open("/dev/phantom");
    assert(fd > 0);

    reg.field1 = 42;
    reg.field2 = 67;

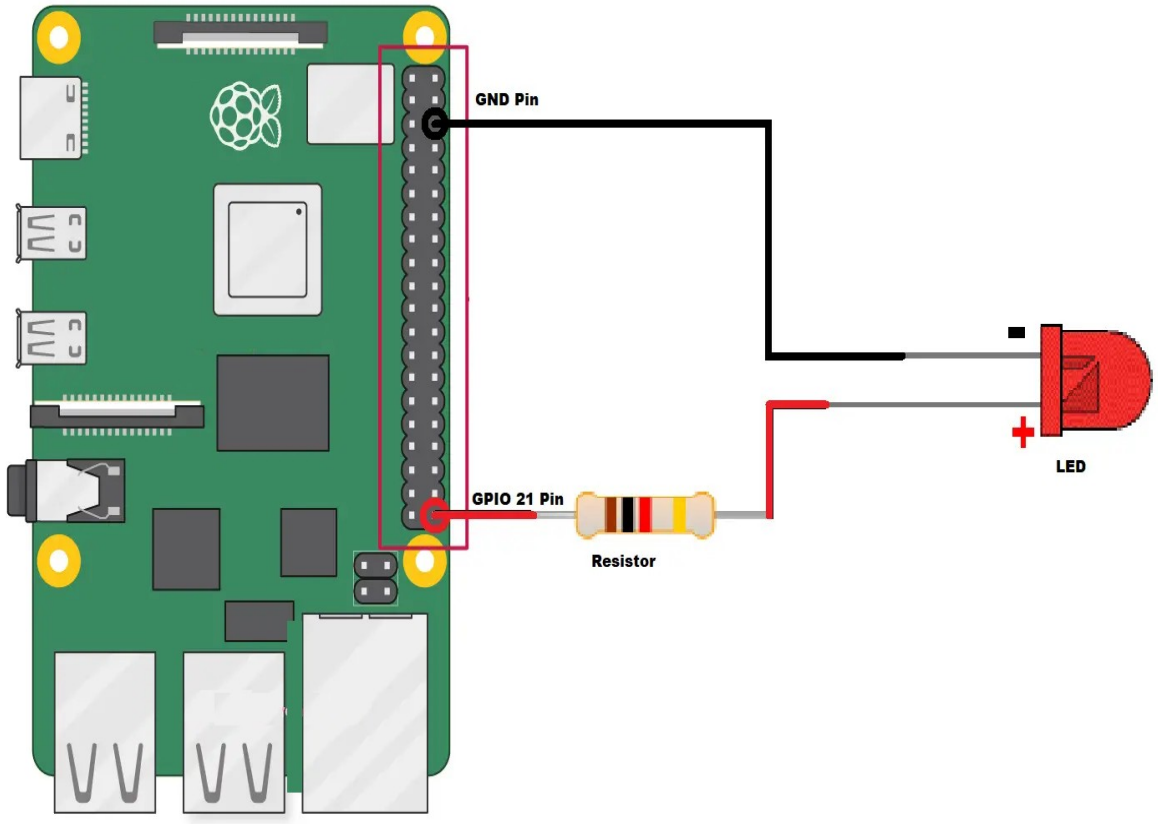
    ret = ioctl(fd, PHN_SET_REG, &reg);
    assert(ret == 0);

    return 0;
}
```

Character Driver – IOCTL function

- The character device driver provides a function ioctl which can be used to communicate with kernel space. Originally this function was used to control the device driver behavior; however it is very common to use this function only for application to kernel communication.

Character Driver – Old Way – LED Driver IOCTL



Thank you